

网络安全技术课程实验报告

实验三



实验名称：基于 OpenSSL 的安全 WEB 服务器实现_____

姓名：____段俊宇_____

学号：____2313815_____

专业：____信息安全、法学双学位班_____

提交日期：2026 年 4 月 11 日_____

一、实验目的

1. 理解 HTTPS 协议及 SSL 协议的工作原理。。
2. 掌握使用 OpenSSL 编程的方法。
3. 提高网络安全系统设计的能力。

二、实验要求及要点

1. 运行于 Linux 平台。
2. 利用 OpenSSL 库编写一个 Web 服务器。
3. 实现 HTTPS 协议下最基本的 GET 命令功能。

三、实验内容

3.1 环境配置

在实验过程中先从 openssl 官网下载源码并编译安装，使用 `pkg-config --modversion openssl` 命令测试，发现得到 1.1.1f，说明安装成功。下面开始正式实验。

3.2 证书生成

首先需要使用 openssl 生成相应的证书，之后才能在代码中进行验证。
生成 CA 证书命令如下：

```
1. # 1. 生成 CA 私钥
2. openssl genrsa -out ca.key 2048
3.
4. # 2. 生成 CA 根证书
5. openssl req -new -x509 -days 3650 -key ca.key -out ca.crt -subj
"/C=CN/ST=Beijing/L=Beijing/O=MyCA/CN=MyRootCA"
```

然后生成服务器证书，由 CA 签名，命令如下：

```
1. openssl genrsa -out server.key 2048
2.
3. # 2. 生成证书签名请求(CSR)
4. openssl req -new -key server.key -out server.csr -subj
"/C=CN/ST=Beijing/L=Beijing/O=MyServer/CN=localhost"
5.
6. # 3. 使用 CA 证书签名服务器证书
```

```
7. openssl x509 -req -days 365 -in server.csr -CA ../ca/ca.crt -CAkey ../ca/ca.key -
set_serial 01 -out server.crt
8.
9. # 4. 验证服务器证书
10. openssl verify -CAfile ../ca/ca.crt server.crt
11.
12. # 5. 创建 PEM 格式文件, 包含证书和私钥
13. cat server.crt server.key > server.pem
```

现在就完成了服务端证书的分发, 之后代码中只要验证该证书即可。

3.3 Web 服务器代码

接下来我将使用注释的方式对代码进行解释。

3.2.1 客户端结构体

本部分基于人工智能生成结果修改。

```
1. // 客户端数据结构
2. typedef struct
3. {
4.     int client_fd;
5.     SSL_CTX *ctx;
6.     struct sockaddr_in client_addr;
7. } client_info_t;
```

3.2.2 获取文件大小

本部分基于人工智能生成结果修改。

```
1. // 获取文件大小
2. long get_file_size(const char *filename)
3. {
4.     struct stat st;
5.     if(stat(filename, &st) == 0)
6.         return st.st_size;
7.     return -1;
8. }
```

3.2.3 获取文件 MIME 类型

本部分基于人工智能生成结果修改。

```
1. const char* get_mime_type(const char *filename)
2. {
3.     const char *ext = strrchr(filename, '.');
4.     if(!ext) return "text/plain";
5.
6.     if(strcmp(ext, ".html") == 0 || strcmp(ext, ".htm") == 0) return "text/html;
charset=UTF-8";
```

```

7.     if(strcmp(ext, ".css") == 0) return "text/css";
8.     if(strcmp(ext, ".js") == 0) return "application/javascript";
9.     if(strcmp(ext, ".jpg") == 0 || strcmp(ext, ".jpeg") == 0) return "image/jpeg";
10.    if(strcmp(ext, ".png") == 0) return "image/png";
11.    if(strcmp(ext, ".gif") == 0) return "image/gif";
12.
13.    return "text/plain";
14. }

```

该部分代码对不同文件类型进行标注，便于在后面进行处理。

3.2.4 初始化 SSL 上下文

本部分基于人工智能生成结果修改。

```

1. SSL_CTX* init_ssl_context()
2. {
3.     SSL_library_init();
4.     SSL_load_error_strings();
5.     OpenSSL_add_all_algorithms();
6.
7.     const SSL_METHOD *method = TLS_server_method();
8.     SSL_CTX *ctx = SSL_CTX_new(method);
9.
10.    if(!ctx)
11.    {
12.        ERR_print_errors_fp(stderr);
13.        return NULL;
14.    }
15.    // 加载服务器证书
16.    if(SSL_CTX_use_certificate_file(ctx, "../server/server.crt", SSL_FILETYPE_PEM) <=
17.0)
18.    {
19.        ERR_print_errors_fp(stderr);
20.        return NULL;
21.    }
22.    // 加载服务器私钥
23.    if(SSL_CTX_use_PrivateKey_file(ctx, "../server/server.key", SSL_FILETYPE_PEM) <=
24.0)
25.    {
26.        ERR_print_errors_fp(stderr);
27.        return NULL;
28.    }
29.    // 检查私钥是否匹配证书
30.    if(!SSL_CTX_check_private_key(ctx))
31.    {
32.        fprintf(stderr, "Private key does not match certificate!\n");
33.    }
34.    return ctx;
35. }

```

```

31.     return NULL;
32. }
33.
34. printf("SSL Context initialized successfully\n");
35. return ctx;
36. }

```

该部分代码从 server 文件夹中加载了服务端的证书和密钥，并且检查是否匹配，实现了安全 Web 服务。

3.2.5 处理 HTTP 请求

本部分基于人工智能生成结果修改。

```

1. void handle_request(SSL *ssl, const char *request)
2. {
3.     char method[16] = {0};
4.     char path[256] = {0};
5.     char version[16] = {0};
6.     // 解析请求行
7.     sscanf(request, "%s %s %s", method, path, version);
8.     printf("Method: %s, Path: %s\n", method, path);
9.     // 默认首页
10.    if(strcmp(path, "/") == 0)
11.    {
12.        strcpy(path, "/index.html");
13.    }
14.    // 构建文件路径
15.    char filepath[512] = "WWW";
16.    strcat(filepath, path);
17.    printf("Looking for file: %s\n", filepath);
18.    // 检查文件是否存在
19.    long file_size = get_file_size(filepath);
20.    FILE *fp = fopen(filepath, "rb");
21.
22.    if(fp && file_size > 0)
23.    {
24.        // 读取文件内容
25.        char *content = (char*)malloc(file_size);
26.        if(content)
27.        {
28.            fread(content, 1, file_size, fp);
29.            fclose(fp);
30.            // 发送 HTTP 响应头
31.            char header[BUFFER_SIZE];
32.            const char *mime_type = get_mime_type(filepath);

```

```

33.         sprintf(header,
34.             "HTTP/1.1 200 OK\r\n"
35.             "Content-Type: %s\r\n"
36.             "Content-Length: %ld\r\n"
37.             "Connection: close\r\n"
38.             "\r\n",
39.             mime_type, file_size);
40.
41.         SSL_write(ssl, header, strlen(header));
42.         SSL_write(ssl, content, file_size);
43.
44.         printf("Sent: %s (%ld bytes)\n", filepath, file_size);
45.         free(content);
46.     }
47.     else
48.     {
49.         const char *error = "HTTP/1.1 500 Internal Error\r\n\r\n";
50.         SSL_write(ssl, error, strlen(error));
51.     }
52. }
53. else
54. {
55.     // 文件不存在, 返回 404
56.     const char *not_found =
57.         "HTTP/1.1 404 Not Found\r\n"
58.         "Content-Type: text/html\r\n"
59.         "\r\n"
60.         "<html><body><h1>404 Not Found</h1><p>File not found</p></body></html>";
61.     SSL_write(ssl, not_found, strlen(not_found));
62.     printf("404 Not Found: %s\n", filepath);
63.
64.     if(fp) fclose(fp);
65. }
66. }

```

这部分是整个代码的核心，首先解析请求行内容，接下来构建 html 文件的路径并检查文件是否存在。当文件存在时，读取文件内容，发送 HTTP 响应头并传输文件内容；如果文件不存在，则显示 404 界面。这样实现了完整的请求处理逻辑。

3.2.6 客户端线程

本部分基于人工智能生成结果修改。

1. // 客户端线程函数

```

2. void* client_thread(void* arg)
3. {
4.     client_info_t *info = (client_info_t*)arg;
5.
6.     printf("Thread %lu handling connection from: %s:%d\n",
7.           pthread_self(),
8.           inet_ntoa(info->client_addr.sin_addr),
9.           ntohs(info->client_addr.sin_port));
10.    // 创建 SSL 对象
11.    SSL *ssl = SSL_new(info->ctx);
12.    SSL_set_fd(ssl, info->client_fd);
13.    // SSL 握手
14.    if(SSL_accept(ssl) <= 0)
15.    {
16.        ERR_print_errors_fp(stderr);
17.    }
18.    else
19.    {
20.        char buffer[BUFFER_SIZE] = {0};
21.        int bytes = SSL_read(ssl, buffer, BUFFER_SIZE - 1);
22.
23.        if(bytes > 0)
24.        {
25.            printf("\n--- Request from thread %lu ---\n", pthread_self());
26.            printf("%s\n", buffer);
27.            printf("--- End Request ---\n\n");
28.
29.            handle_request(ssl, buffer);
30.        }
31.    }
32.    // 清理
33.    SSL_shutdown(ssl);
34.    SSL_free(ssl);
35.    close(info->client_fd);
36.    free(info);
37.
38.    printf("Thread %lu connection closed\n", pthread_self());
39.
40.    return NULL;
41. }

```

该部分代码实现了客户端逻辑，对每个客户端都单独创建线程并处理请求，从而实现服务端的并发处理请求。

3.2.7 main 函数

本部分基于人工智能生成结果修改。

```
1. int main()
2. {
3.     // 忽略 SIGPIPE 信号
4.     signal(SIGPIPE, SIG_IGN);
5.     // 初始化 SSL
6.     SSL_CTX *ctx = init_ssl_context();
7.     if(!ctx)
8.     {
9.         fprintf(stderr, "Failed to initialize SSL\n");
10.        return -1;
11.    }
12.    // 创建 socket
13.    int sockfd = socket(AF_INET, SOCK_STREAM, 0);
14.    if(sockfd < 0)
15.    {
16.        perror("socket");
17.        return -1;
18.    }
19.    // 设置 socket 选项, 允许端口重用
20.    int opt = 1;
21.    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
22.    // 绑定地址
23.    struct sockaddr_in addr;
24.    addr.sin_family = AF_INET;
25.    addr.sin_port = htons(PORT);
26.    addr.sin_addr.s_addr = INADDR_ANY;
27.    if(bind(sockfd, (struct sockaddr*)&addr, sizeof(addr)) < 0)
28.    {
29.        perror("bind");
30.        return -1;
31.    }
32.    // 监听
33.    if(listen(sockfd, BACKLOG) < 0)
34.    {
35.        perror("listen");
36.        return -1;
37.    }
38.
39.    printf("\n===== \n");
40.    printf("HTTPS Server Started (Concurrent)\n");
41.    printf("Port: %d\n", PORT);
42.    printf("Access: https://localhost:%d\n", PORT);
43.    printf("===== \n\n");
```



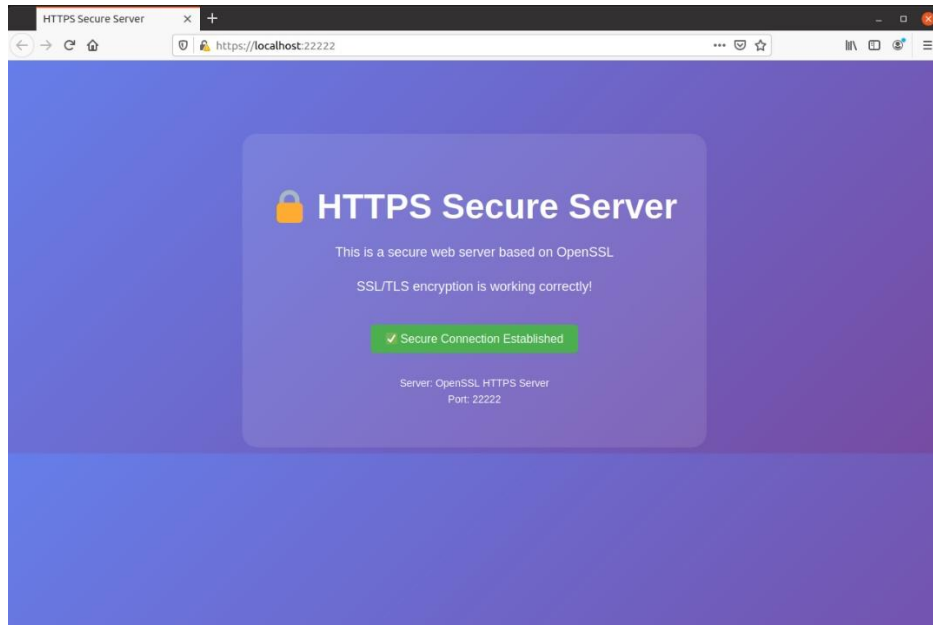
```

44.
45.     // 为每个请求创建新线程
46.     while(1)
47.     {
48.         struct sockaddr_in client_addr;
49.         socklen_t client_len = sizeof(client_addr);
50.
51.         int client_fd = accept(sockfd, (struct sockaddr*)&client_addr, &client_len);
52.         if(client_fd < 0)
53.         {
54.             perror("accept");
55.             continue;
56.         }
57.
58.         // 创建客户端信息结构
59.         client_info_t *info = (client_info_t*)malloc(sizeof(client_info_t));
60.         info->client_fd = client_fd;
61.         info->ctx = ctx;
62.         memcpy(&info->client_addr, &client_addr, sizeof(client_addr));
63.
64.         // 创建线程处理请求
65.         pthread_t tid;
66.         if(pthread_create(&tid, NULL, client_thread, info) != 0)
67.         {
68.             perror("pthread_create");
69.             close(client_fd);
70.             free(info);
71.         }
72.         else
73.         {
74.             pthread_detach(tid); // 分离线程，自动回收资源
75.         }
76.     }
77.
78.     close(sockfd);
79.     SSL_CTX_free(ctx);
80.
81.     return 0;
82. }

```

这部分是代码的主函数，实现了服务端的全部流程，首先初始化 SSL、创建 SOCKET 套接字、绑定地址并开始监听，当有新请求时为其创建新线程，处理后回收资源，实现线程的并发处理。

使用 `gcc -o server main.c -lssl -lcrypto -lpthread` 命令编译代码，再使用 `server` 命令运行 ELF 文件，这时终端输出了成功初始化的日志。在浏览器中输入 `https://localhost:22222`，可以访问 `index.html` 界面，多个标签页都可以访问，如下所示：



```
[04/11/26]seed@VM:~/.../Code$ server
SSL Context initialized successfully

=====
HTTPS Server Started (Concurrent)
Port: 22222
Access: https://localhost:22222
=====

Thread 140067658823424 handling connection from: 127.0.0.1:45924

--- Request from thread 140067658823424 ---
GET / HTTP/1.1
Host: localhost:22222
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1

--- End Request ---

Method: GET, Path: /
Looking for file: WWW/index.html
Sent: WWW/index.html (1379 bytes)
Thread 140067658823424 connection closed
Thread 140067658823424 handling connection from: 127.0.0.1:45940

--- Request from thread 140067658823424 ---
GET / HTTP/1.1
Host: localhost:22222
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:83.0) Gecko/20100101 Firefox/83.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Upgrade-Insecure-Requests: 1

--- End Request ---

Method: GET, Path: /
Looking for file: WWW/index.html
Sent: WWW/index.html (1379 bytes)
Thread 140067658823424 connection closed
```

此外我还发现如果在浏览器地址栏中输入 `http://localhost:22222`, 发现无法连接, 这说明成功实现安全的 Web 服务器。

四、关于人工智能使用的说明

4.1 人工智能使用声明

本人段俊宇承诺: 对此文档的所有内容正确性、真实性负责, 充分理解了提交报告的知识内容, 并对使用大模型辅助的内容进行了正确的标注。

4.2 人工智能使用标注

在本次实验中, 环境配置和实验代码在人工智能的帮助下完成, 其中一些问题由人工智能进行解答。对于人工智能生成的代码我进行了一些修改, 然后呈现出了更好的展示结果。

4.3 人工智能技术使用总结

我使用 `deepseek` 和 `chatgpt` 辅助我完成本次实验, 使用过程和第二次实验一样, `deepseek` 可以很好的完成实验指导任务, 步骤和命令都很详细。但是 `deepseek` 生成的代码有些问题, 而 `chatgpt` 生成的代码一次就可以直接运行, 更加节省时间。